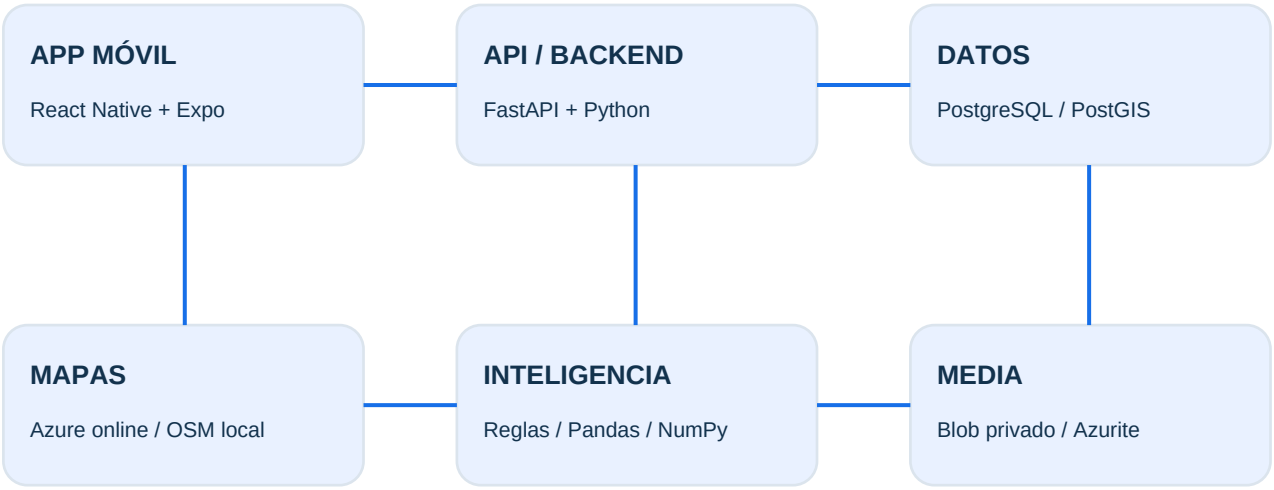
SafeRouteAI

Manual técnico y de usuario

Versión 1.0.0 · Salamanca, Guanajuato



Rutas, reportes comunitarios y alertas en una aplicación móvil.

Código, configuración, reglas de riesgo, uso cotidiano y trazabilidad de requisitos.

La aceptación final con Azure, PostGIS y dispositivos se documenta por separado de las pruebas automáticas.

SafeRouteAI combina navegación e información comunitaria para comparar desplazamientos en Salamanca, Guanajuato. La primera versión incluye automóvil y caminata, Android e iOS, reportes con evidencia opcional, corroboración y continuidad de recorridos cuando la red se interrumpe.

Este manual describe el código entregado. La validación de bundles y pruebas automáticas no sustituye la validación de mapas con credenciales reales, la base PostGIS ni los dispositivos móviles.

1. Arquitectura

La app Expo obtiene coordenadas con permiso del usuario y consume la API REST `/api/v1` mediante JSON. FastAPI autentica las operaciones con JWT, coordina la lógica de negocio, consulta cartografía y rutas de Azure Maps y utiliza PostgreSQL/PostGIS para persistencia y proximidad espacial. Azure Blob guarda fotografías privadas; la base conserva sus referencias. Azurite proporciona el equivalente local.

Las subsecciones de backend son usuarios, ubicaciones, rutas, preferencias, incidentes, evaluación, recomendaciones, alertas y administración. Autenticación, API, datos, media, servicios externos e inteligencia están separados. NumPy y Pandas procesan factores y agregaciones. Scikit-learn queda previsto para una evolución con datos representativos; no se carga ningún modelo sintético.

El mapa online se presenta mediante Azure Maps Web SDK en WebView. Su proxy restringe las solicitudes a recursos de cartografía de Azure; la API KEY permanece en el backend. El mapa offline utiliza calles OSM en GeoJSON y Leaflet empaquetado dentro de la app. Los dos mapas representan coordenadas WGS84.

2. Instalación local

Instala Python 3.12 o superior, Node 22 o superior y Docker Desktop con contenedores Linux. Android requiere Android Studio/SDK; un build iOS local requiere macOS y Xcode. El frontend utiliza Expo SDK 57 y React Native 0.86.3, Android 7.0+ (API 24) e iOS 16.4+. Instala las versiones bloqueadas, no actualices bibliotecas nativas por separado.

Desde `app`, ejecuta:

```
python -m venv .venv
.venv/Scripts/python.exe -m pip install -r backend/requirements.lock.txt
./infrastructure/setup.ps1
docker compose -f infrastructure/compose.yaml up -d
.venv/Scripts/python.exe -m alembic -c backend/alembic.ini upgrade head
.venv/Scripts/python.exe -m backend.cli seed
.venv/Scripts/python.exe -m backend.cli prepare-osm
.venv/Scripts/python.exe -m uvicorn backend.main:app --host 0.0.0.0 --port 8000
```

La clave Azure se introduce personalmente en `app/backend/.env`, variable `AZURE_MAPS_API_KEY`. La plantilla `backend/.env.example` la identifica explícitamente. Alternativamente configura el modo `sas` y `AZURE_MAPS_SAS_TOKEN`. Nunca coloques estos valores en variables `EXPO_PUBLIC_*`.

En el frontend, copia `.env.example` a `.env` y reemplaza `EXPO_PUBLIC_API_URL` por la IP de tu backend. En un teléfono físico, `localhost` apunta al propio teléfono. Comparte la red Wi-Fi con la computadora.

```
cd frontend
npm ci
npm run android
# macOS con Xcode:
npm run ios
# Para un build de desarrollo ya instalado:
npm start
```

El script de configuración genera JWT_SECRET y la conexión del emulador local. No sobrescribe secretos existentes. La migración crea las tablas y la extensión PostGIS; seed crea únicamente categorías reales de clasificación, no reportes ni usuarios ficticios. prepare-osm descarga redes independientes para auto y caminata. Se reutilizan mientras no se retiren o actualicen sus archivos GraphML.

3. Cuentas y recuperación

Selecciona Crear una cuenta, introduce nombre, correo y contraseña de al menos diez caracteres. Guarda el código privado que aparece una sola vez. La app no almacena ese código en texto claro. Para recuperar acceso, introduce correo, código privado y nueva contraseña. El código se invalida, las sesiones anteriores se revocan y se emite otro código que también debes conservar.

Perfil permite modificar nombre, transporte, tolerancia de riesgo, horario silencioso, alertas y tema. Los cambios se guardan en el servidor. Para cerrar sesión, conecta la app y envía primero los reportes pendientes; así se revocan la sesión y el registro de notificaciones. La cuenta administradora se crea por consola mediante `python -m backend.cli create-admin`, sin credenciales predeterminadas.

4. Mapas y recorridos

Activa Mi ubicación y acepta el permiso. Puedes buscar una dirección o tocar un destino en el mapa. Calcula rutas para ver distancia, duración, índice estimado y factores. Selecciona una alternativa y usa Ver factores de seguridad para consultar los incidentes que influyeron en su evaluación.

Iniciar recorrido conserva geometría e instrucciones. La app muestra siguiente maniobra, distancia restante, tiempo orientativo y aviso si sales del recorrido. El seguimiento es únicamente en primer plano. Finalizar recorrido elimina la ruta activa guardada. El historial conserva las consultas; una ruta histórica debe recalcularse con datos actuales antes de volver a iniciarse.

Cuando se recibe un cambio de incidentes, la ruta activa se evalúa nuevamente. Un aumento relevante o una nueva situación crítica genera aviso. Consultar alternativas calcula desde la ubicación actual; el usuario debe elegir explícitamente el cambio. Si no existe alternativa adecuada, se recomienda evitar continuar hacia la zona afectada y detenerse únicamente en un lugar que pueda verificar seguro.

Las alternativas Azure se complementan con un corredor de menor exposición obtenido sobre OSM. Los puntos intermedios se envían a Azure y la geometría devuelta se evalúa nuevamente; no se asume que Azure implementa un parámetro de evasión de incidentes. Se deduplican rutas con geometría equivalente.

5. Reportes, fotografías y corroboración

Toca Reportar, comprueba la ubicación, selecciona categoría y describe lo observado. Las categorías cubren robos/asaltos, enfrentamientos, incendios, accidentes, manifestaciones, retenes, operativos, bloqueos, inundaciones, obstáculos, deterioro vial e iluminación adecuada, deficiente o inexistente. Puedes seleccionar una foto o usar la cámara; la foto es opcional y nunca bloquea un reporte sin ella.

El backend valida imágenes JPEG, PNG o WebP de hasta 8 MB y 24 megapíxeles, corrige orientación, reduce tamaño y elimina EXIF/ubicación. Se guardan blobs privados y se consultan mediante la API autenticada. Si falla la foto, el reporte permanece enviado y la app puede reintentar solo la evidencia.

Para corroborar, abre un incidente y responde Continúa, Finalizó o No puedo confirmar. El servidor exige ubicación de menos de 120 segundos, precisión de hasta 100 m y distancia de hasta 500 m. No puedes corroborar tu propio reporte, votar dos veces sobre la misma versión ni corroborar eventos críticos. Dos respuestas independientes coincidentes confirman o finalizan el incidente. Una confirmación positiva renueva la vigencia; una respuesta No puedo confirmar no incrementa confianza.

Administración permite suspender/reactivar usuarios, confirmar/finalizar/ocultar reportes y editar categorías. Esas operaciones se autorizan por rol en el servidor y quedan en el registro de auditoría. Un usuario normal no obtiene permisos administrativos aunque modifique la interfaz móvil.

6. Índice y reglas de riesgo

Las reglas se centralizan en `backend/intelligence/risk_algorithm/rules.json`, versión 1.0.0. Son valores iniciales configurables para comparar exposición reportada, sin calibración probabilística. La ausencia de reportes no demuestra seguridad; se muestra Información insuficiente cuando no hay factores aplicables. Un reporte de iluminación adecuada tampoco demuestra ausencia de delitos.

Para cada factor cercano al recorrido, se calcula una contribución con severidad de categoría, peso de grupo, confianza, vigencia y distancia. Radio de influencia: 300 m. Pesos de grupo: eventos agudos 1.0, vialidad 0.7 e iluminación 0.6. Confianza inicial: 0.5; corroborado: 1.0. La iluminación deficiente se aplica de 19:00 a 07:00, hora de Ciudad de México.

La contribución activa decae exponencialmente con antigüedad respecto de la vigencia de la categoría. El histórico de hasta 90 días se conserva con peso reducido de 0.15 y no genera alertas activas. La proximidad reduce linealmente la contribución hasta cero a 300 m. La normalización utiliza `100 * (1 - exp(-suma_contribuciones / 1.5))`; el índice de seguridad es su complemento, cuando existe información aplicable. La evaluación conserva versión, fecha y contribuciones principales.

Vigencias iniciales: agudos 6 horas; inundaciones, bloqueos y obstáculos 24 horas; iluminación y deterioro vial 720 horas. Las categorías son editables por administrador. Un riesgo crítico ocurre al pasar a 100 m de un evento crítico vigente, o al superar 75 puntos. Desde 35 puntos se etiqueta exposición elevada. Los incidentes ocultos no influyen; los vencidos/finalizados solo aportan histórico.

La tolerancia controla el desvío máximo respecto del tiempo más corto: baja 2.0, media 1.6, alta 1.3. Se priorizan rutas no críticas con menor índice conocido dentro de ese límite. Cuando falta información en alguna alternativa se comunica explícitamente. Nunca se presenta el índice como garantía.

7. Offline, alertas y privacidad

En Perfil, descarga el sector de 2 km alrededor de tu ubicación. El máximo es 50 MB y la descarga se verifica mediante SHA-256 antes de guardarse. Puedes actualizarlo o eliminarlo. Contiene calles y nombres OSM recortados a ese radio; no contiene teselas de Azure. Puedes usar el botón de mapa descargado para ahorrar datos manteniendo conexión para reportes y alertas. El motor local no calcula rutas nuevas: conserva el mapa, GPS e instrucciones de la ruta que ya habías iniciado.

Los reportes offline conservan UUID, hora original y foto local. Al reconectar, se envían con la misma identidad; repetir una solicitud no crea otro reporte. La evidencia puede reintentarse por separado. Los reportes atrasados cuya vigencia ya terminó ingresan como vencidos, sin revivir una alerta antigua. Actividad muestra la cola pendiente y permite reintentar o descartar un envío local fallido. Puedes continuar sin una fotografía pendiente; un reporte ya recibido por el servidor permanece registrado. Los errores de validación requieren revisar el reporte; no se convierten en envíos exitosos.

Las actualizaciones visibles utilizan WebSocket con ticket de un solo uso. Las notificaciones push se activan desde Perfil mediante permisos del sistema y credenciales del proyecto Expo/APNs/FCM. Solo se generan por ubicación reciente o una ruta asociada; la app no rastrea continuamente en segundo plano. Las notificaciones no son garantía de entrega inmediata y no existen alertas nuevas offline. El trabajador conserva intentos y consulta recibos de Expo para conocer el resultado.

Las ubicaciones se eliminan al ocultar la app cuando la red lo permite y, en el servidor, después de diez minutos. No hay historial público de posiciones de usuarios. Contraseñas y códigos usan Argon2; acceso JWT dura 15 minutos y refresh rota con vigencia de 30 días. Logout, recuperación y suspensión revocan sesiones. Los secretos no se incluyen en contratos, errores ni documentación.

8. API, mantenimiento y despliegue

La documentación interactiva está en `/docs`. El contrato exportado está en `docs/openapi.json`. Los recursos incluyen auth, users/me, preferences, locations, search, routes, incidents, categories, corroborations, photos, alerts, history, devices, offline-packages, realtime y admin. Usan fechas UTC, GeoJSON longitud/latitud y respuestas HTTP 401/403/404/409/422/429/502/503/504 para errores controlados.

Tras modificar schemas ejecuta `python -m backend.cli export-openapi` y, desde frontend, `npm run generate:api`. Los tipos generados se usan para los DTO de la aplicación. El generador OpenAPI se ejecuta en un entorno temporal de TypeScript 5 para evitar conflictos con TypeScript 6.

En producción configura HTTPS, `ENVIRONMENT=production`, `APP_ENV=production`, una base PostGIS, Blob privado, claves y contraseñas propias, CORS exacto y copias de seguridad. El backend valida que producción tenga HTTPS y PostgreSQL. Nunca uses las credenciales del emulador fuera de desarrollo. El contenedor corre sin root. La publicación de APK/IPA requiere tus cuentas y credenciales.

Usa un único worker de API: WebSocket, tickets y límites de frecuencia son locales al proceso. Antes de múltiples réplicas añade coordinación compartida. `/health` informa vida y `/ready` consulta DB. Supervisa errores, latencia, entregas fallidas, disponibilidad de Azure y actualizaciones de paquetes.

9. Validación y límites de entrega

Se verifican automáticamente recuperación y revocación, idempotencia, permisos, caducidad, corroboración cercana, sanitización fotográfica, contratos Azure, corredores dirigidos, recomendación, generación offline, ausencia de datos y aplicación nocturna de iluminación. En móvil se comprueban reintentos de evidencia y progreso de navegación. También se ejecuta TypeScript y exportación Hermes para Android/iOS. Estas verificaciones no equivalen a una prueba en teléfonos reales.

Pendientes de entorno: migraciones y consultas con PostGIS, operaciones con Azurite/Blob, Azure Maps con tu clave, push real y aceptación de interfaz/ubicación en Android e iOS. La prueba de integración requiere `TEST_DATABASE_URL` de una base aislada. La matriz anexa distingue código y aceptación.

Los objetivos de actualización en cinco segundos y rutas en cinco segundos requieren medición con servidor, red, datos y Azure configurados. No se afirma que ya hayan sido alcanzados. La consulta de ubicación usa un intervalo nominal de cinco segundos sujeto al sistema operativo.

La revisión npm identifica avisos transitivos de Expo/Metro/xcode. Su propuesta automática implica retroceder Expo/React Native a versiones incompatibles; no se aplica esa sustitución. Se conserva el reporte y debe revisarse con actualizaciones compatibles antes de publicar la aplicación.

10. Fuentes técnicas

Azure Maps Route Directions: Consultar documentación oficial

Azure Maps Web SDK: Consultar documentación oficial

Restricción offline Azure: Consultar documentación oficial

Azurite: Consultar documentación oficial

Expo ubicación: Consultar documentación oficial

Expo notificaciones: Consultar documentación oficial

Waze reportes: Consultar documentación oficial

Google Maps controles: Consultar documentación oficial

OpenStreetMap atribución: Consultar documentación oficial

11. Matriz de requisitos

Código implementado y evidencia de validación. Pendiente significa que se necesita un servicio externo, credencial o dispositivo; no equivale a una prueba aprobada.

ID	Requerimiento	Implementacion	Validacion
RF-01	Registrar cuenta	auth/register y pantalla de registro	Prueba API
RF-02	Iniciar sesion	auth/login y sesiones JWT	Prueba API y dispositivo pendiente
RF-03	Recuperar contraseña	auth/recover con codigo rotatorio	Prueba API
RF-04	Gestionar perfil	users/me y Perfil	Prueba contrato; dispositivo pendiente
RF-05	Mapa y ubicacion	Azure Web SDK y GPS foreground	Bundle; Azure y dispositivo pendientes
RF-06	Buscar destino	Azure geocode y seleccion de mapa	Credencial Azure pendiente
RF-07	Rutas alternativas	Azure Directions y corredor OSM auto/pie	Contrato y deduplicacion; Azure pendiente
RF-08	Indice estimado	risk_algorithm versionado	Pruebas de riesgo
RF-09	Comparar seguridad	Panel de rutas con factores	Bundle; dispositivo pendiente
RF-10	Recomendar menor exposicion	Motor de recomendaciones	Prueba de rechazo critico
RF-11	Registrar incidente	incidents y cola UUID	Prueba idempotencia
RF-12	Evidencia fotografica	Foto opcional Blob con sanitizacion	Prueba EXIF y cola; Blob pendiente
RF-13	Solicitar corroboracion	Alertas por proximidad y precision	Prueba API; push pendiente
RF-14	Corroborar vigencia	Continua/Finalizo/No puedo confirmar	Prueba independiente y cercana
RF-15	Actualizar estados	Votos por version y caducidad	Pruebas API y expiracion
RF-16	Mostrar alertas	WebSocket y actividad con push	Codigo y bundle; dispositivo pendiente
RF-17	Eventos de ruta	Categorias y proximidad a recorrido	Codigo; flujo externo pendiente
RF-18	Actualizar incidentes	Versionado y reevaluacion de recorrido	Pruebas de estados; dispositivo pendiente
RF-19	Mapa de calor	Agregacion historica Pandas y capas	Codigo; dispositivo pendiente
RF-20	Iluminacion	Tres categorias y ponderacion nocturna	Prueba nocturna
RF-21	Vialidad	Categorias de condiciones de via	Seed y contratos
RF-22	Factores integrados	Severidad/proximidad/edad/confianza/grupo	Pruebas de riesgo
RF-23	Cambios en recorrido	Reevaluacion al recibir cambios	Bundle; escenario en dispositivo pendiente

ID	Requerimiento	Implementacion	Validacion
RF-24	Cambio de ruta	Alternativas y seleccion explicita	Contrato de rutas; dispositivo pendiente
RF-25	Detencion preventiva	Advertencia sin alternativa adecuada	Prueba de riesgo critico
RF-26	Administracion	Rol servidor y pantalla administrativa	Prueba permisos y categorias
RNF-01	Proteger informacion	Argon2 JWT refresh y autorizacion	Pruebas revocacion; HTTPS produccion pendiente
RNF-02	Proteger ubicacion	Permiso foreground y borrado por retencion	Codigo; dispositivo pendiente
RNF-03	Disponibilidad	Timeouts health ready y errores controlados	Prueba falta de Azure; despliegue pendiente
RNF-04	Actualizacion oportuna	WebSocket y jobs de caducidad	Prueba estados; medicion 5s pendiente
RNF-05	Rendimiento	Indices espaciales y solicitudes paralelas	DDL y paquete real; benchmark pendiente
RNF-06	Usabilidad	Mapa principal acciones y estados	Bundle; aceptacion de usuario pendiente
RNF-07	Compatibilidad movil	Expo Android/iOS y safe areas	Bundles Android/iOS; dispositivos pendientes
RNF-08	Crecimiento	Modulos repositorios y configuracion	Revision de arquitectura
RNF-09	Integridad	Propietario rol UUID y votos independientes	Pruebas API
RNF-10	Explicar indice	Factores version fecha y datos insuficientes	Pruebas y pantalla de factores